# VEIL — Phase 4 Demo Lock Kit

Locked Demo Script · Pre-Demo Checklist · Pitch Deck Outline

ISRO SIH: On-device Visual Perception for Light-weight Browser Agents

September 2026

## Contents

# 1 0. What this document is for

Per the module spec's Phase 4 ("hardening for the demo"): lock the demo script to one flawless flow, re-run the benchmark harness, capture final numbers for the pitch deck, rehearse the side-by-side comparison screen. This document is the three artifacts that phase actually produces — the script, the checklist, and the deck outline — so nothing is being improvised on presentation day.

This is intentionally decoupled from the extension's source code — it doesn't require the real `detector.js`/`content.js`/server files to be correct, only for the demo *sequence* and *talking points* to be fixed in advance. Fill in the bracketed [ ] placeholders with your real numbers once the benchmark harness (with the adversarial fixtures merged, per the previous deliverable) has actually run against your build.

# 2 1. Locked demo script (five minutes, word-for-word)

Matches PRD Section 11's six-step structure, expanded to exact spoken lines and exact on-screen actions. Rehearse this verbatim at least twice before presenting — per the module spec's Phase 4 exit criteria: "the five-minute demo script runs flawlessly, twice in a row, on the actual demo machine."

## 2.1 Timing budget

| Step | Elapsed | Duration |
|---|---|---|
| 1. Open the mock checkout page | 0:00 | 20s |
| 2. Show the dashboard, counters at zero | 0:20 | 25s |
| 3. Type the task instruction | 0:45 | 20s |
| 4. Side-by-side view, counters ticking up | 1:05 | 90s |
| 5. Server action + execution | 2:35 | 90s |
| 6. Benchmark numbers slide | 4:05 | 55s |
| **Total** | | **5:00** |

If a step runs long during rehearsal, cut time from Step 4 or 5 (the live-watching beats), never from Step 6 — the numbers slide is the evidence, per the module spec's own framing ("that last slide is doing as much work as the live demo").

## 2.2 Step-by-step script

**Step 1 — Open the mock checkout page (0:00–0:20)**

> "This is a mock checkout page — name, email, card number, CVV, address, and a Place order button. Nothing special about it; it's what any e-commerce form looks like."

*Action:* Tab already open, page loaded, extension icon visible in the toolbar. Do not open the extension yet.

**Step 2 — Show the dashboard, counters at zero (0:20–0:45)**

> "Before I do anything, here's VEIL's Privacy Observatory — it's already running. Right now it's scanned nothing, so every counter reads zero."

*Action:* Click the toolbar icon or open the side panel. Point at the three top counters (fields redacted, PII types blocked, sensitivity level) and the empty latency bar.

**Step 3 — Type the task instruction (0:45–1:05)**

> "I'll tell the agent what I want: 'complete this purchase.' That's the only instruction it gets — everything else it has to figure out from the page itself."

*Action:* Type the instruction into the popup's task input field, press Run/Enter. Do not narrate over the text appearing — let it be read.

**Step 4 — Side-by-side view, counters ticking up (1:05–2:35)**

> "Watch the right-hand pane. That's not the real page — it's the sanitized version, the only thing that ever leaves this device. As VEIL scans, you'll see the name, email, card number, and CVV fields go dark on the right, while the left pane — the real page — stays untouched. The counters are counting in real time, not after the fact."

*Action:* Side-by-side view is the visual focus for this entire 90-second window. Let it run without over-narrating — this is the "killer demo moment" the module spec calls out; the visual should carry it, not the talking.

**Step 5 — Server action + execution (2:35–4:05)**

> "VEIL sent that sanitized version — structure and labels only, no real values — to the model, and asked for one next action. Here's what came back: [read the actual `targetDescription`, e.g. 'click the button labeled Place order']. Notice it's a description, not pixel coordinates — that's deliberate, coordinates break the moment the page reflows or the screen resolution changes. VEIL resolves that description to the real button on the real page and clicks it."

*Action:* Show the dashboard's debug panel briefly (the `reasoning` field from the last `ActionTarget`) if it renders cleanly; skip it if it's visually noisy — the resolved click executing on the live page is the important beat, not the JSON.

**Step 6 — Benchmark numbers slide (4:05–5:00)**

> "None of this is 'looks fast' — here are the actual numbers, measured, not estimated."

*Action:* Cut to the pitch deck's numbers slide (Section 3.5, below). Read the five rubric-aligned numbers in order: recall, precision, leakage, memory, latency. End on this slide — do not return to the live demo after it.

# 3 2. Pre-demo checklist

Run this checklist in full at least once on the actual presentation machine, not just a dev machine — per the module spec's Risks table, a cold environment can behave differently than a warm dev setup.

## 3.1 A. Benchmark and numbers (do this first — everything else depends on it)

- ☐ Adversarial fixtures (09–13) merged into the real `ground-truth.json` via `merge-adversarial-fixtures.js`.
- ☐ `npm run benchmark` executed fresh, output captured to a file, not read from memory or an old run.
- ☐ Per-type precision/recall reviewed — any number that dropped from the original 100%/100% is understood (which fixture caused it, why), not just noted.
- ☐ Final numbers copied into the pitch deck's numbers slide (Section 3.5) — no placeholder values left in the deck.
- ☐ Leakage metric re-run against the real redaction pipeline (requires Module 2 complete, per the build spec's Phase 3 note) — zero-leakage claim verified, not assumed.

## 3.2 B. Live pipeline on the demo machine

- ☐ Extension loaded via **Load unpacked** on the actual presentation machine (not just the dev machine) — Chrome profile, extensions list, and permissions confirmed clean.
- ☐ Mock checkout page opens correctly, all six fields render as expected.
- ☐ Full round-trip (capture → detect → redact → server call → action executed) run at least 3 times back-to-back with no manual intervention, no console errors.
- ☐ Chrome Task Manager (`Shift+Esc`) checked during a live run — extension process under the 300MB budget from PRD Section 8.
- ☐ End-to-end latency read from the dashboard's own instrumentation, not a stopwatch — confirmed under or near the 300ms budget, and if over, the gap is understood (which stage — capture, network, VLM — is the bottleneck) so it can be spoken to honestly if asked.
- ☐ If using a local Ollama backend: confirm it's warm (one throwaway call made before the actual demo) — a cold local model load can blow the latency budget on the very first live call.
- ☐ If using a hosted backend as a fallback: confirm network access works on the actual presentation network (conference wifi is not always your dev network) — per PRD Section 7's swap plan, know which backend you're using before walking in, don't decide live.

## 3.3 C. The two-in-a-row test

- ☐ Full 5-minute script run twice, back-to-back, with no restart of the extension/browser between runs — the module spec's literal exit criteria for Phase 4. If the second run behaves differently from the first (different latency, different resolved action, a skip where the first run clicked), find out why before presenting, not during.

## 3.4 D. Fallback plan

- ☐ A screen recording of one clean successful run exists as a backup, in case live network/demo-machine conditions fail during the actual presentation slot. Note in your own script where you'd cut to it, and don't present the recording as live if you have to use it — say so.
- ☐ Know, out loud, what you'll say if a live run does show a skip ("action unclear, skipping") instead of a click — this is a documented, intentional safety behavior (PRD Risks table: "a visible skip beats a wrong click"), not a bug, and saying so calmly is a better recovery than looking rattled.

## 3.5 E. Side-by-side view rehearsal specifically

- □ Rehearsed at least once with someone else watching who wasn't looking at the code — confirm the visual "diff" (real page vs. sanitized) reads clearly to someone seeing it cold, per the module spec's explicit call that this view "gets disproportionate polish" and does "the most persuasive work in the room."

# 4 3. Pitch deck outline

Ends on the benchmark numbers slide, per PRD Section 11 step 6 and the build spec's Phase 4 framing. Roughly 8 slides for a 5-minute demo + short pitch context; adjust slide count to your actual time slot, but keep the numbers slide last regardless.

## 4.1 3.1 Slide 1 — Title + one-liner

> **VEIL — privacy firewall for browser agents** "See locally. Reason remotely. Reveal nothing sensitive."

ISRO SIH problem statement name in a subtitle line: *On-device Visual Perception for Light-weight Browser Agents.*

## 4.2 3.2 Slide 2 — The problem, stated plainly

- Browser agents need to see the screen to act on it.
- Today, "seeing the screen" usually means sending a screenshot to a cloud model — including whatever personal data happens to be on it.
- VEIL's premise: most of that data doesn't need to leave the device for the agent to still work.

## 4.3 3.3 Slide 3 — Architecture, one diagram

Reuse the data-flow diagram from the module build spec (Section 9 of that document) — same diagram in both documents avoids the audience having to reconcile two versions of "how it works."

## 4.4 3.4 Slide 4 — Live demo

No content on this slide beyond a title ("Live demo") — this is where you switch to the actual browser window and run Section 1's script.

## 4.5 3.5 Slide 5 — The numbers (rubric-aligned, ends the deck)

Fill in from the real, freshly-run benchmark (checklist item A above) — do not carry over the numbers from an earlier draft of this deck.

| Rubric line | Weight | VEIL's number |
|---|---|---|
| Accuracy of visual context from screen | 25% | [ measured — see note below ] |
| PII detection recall | 20% | [ from benchmark harness, post-adversarial-fixtures ] |
| PII detection precision | 20% | [ from benchmark harness, post-adversarial-fixtures ] |
| Redaction leakage | 20% | [ from leakage metric — target 0% ] |
| Client-side resource utilization | 20% | [ memory reading from Task Manager, checklist item B ] |
| End-to-end latency | 15% | [ dashboard-instrumented reading, checklist item B ] |

*Note on "accuracy of visual context":* this rubric line doesn't have a single clean automated metric the way precision/recall do — speak to it qualitatively (the format-preserving placeholder redaction feature, if built, is your strongest evidence here: the VLM reasons over correctly-shaped fields, not blank boxes) rather than inventing a number that doesn't exist.

## 4.6 3.6 Slide 6 (optional, if time allows) — What we deliberately didn't build, and why

A short slide naming 2–3 items from the "explicitly out of scope" list (build spec Section 12) — Aadhaar/PAN-specific detectors, a policy editor UI, multi-site generalization — framed as scoping discipline, not as gaps. This preempts the "why doesn't it do X" question by answering it before it's asked, and signals the team understood the trade-off rather than ran out of time by accident.

## 4.7 3.7 Slide 7 (optional) — Roadmap

Placeholder redaction / action-graph cache (if not fully built), clipboard exfiltration guard, DPDP Act compliance framing (Feature 5 from the PRD Addendum) — one line each, framed as "the next thing we'd build," not as unfinished work.

## 4.8 3.8 Closing line

> "VEIL doesn't ask you to trust that we redacted your data — it's built so the request never leaves the device if we didn't."

Ties back to the fail-closed kill switch (Feature 1 from the PRD Addendum) if it's built — this line is only honest to say if that guard actually exists in the running build; don't say it on the strength of the idea alone.

# 5 4. What’s still needed before this document is fully usable

This document is structurally complete but has placeholders that only your real, running build can fill in:

- Every `[ ]` bracket in Section 3.5’s numbers table.
- The actual `targetDescription` string your server returns for the demo task, referenced in Step 5 of the script.
- Confirmation of which VLM backend (local Ollama vs. hosted) you’re actually presenting with, per checklist item B.

None of these can be filled in without running your real extension against your real benchmark — this document sequences and scripts the presentation, it doesn’t substitute for the underlying system actually working.